

Глава 6: Исследователь лабиринта

Учим робота проходить лабиринт

В Главе 5 ты научился следовать вдоль стен и перемещаться по комнате. Теперь собираем всё вместе, чтобы исследовать лабиринт!

Что ты изучишь

1. Как устроены ячейки лабиринта (стены и проходы)
 2. Определение открытых стен
 3. Использование `move_to_wall()` для безопасного перемещения между ячейками
 4. Ведение карты посещённых ячеек
 5. Выбор следующей ячейки
 6. Создание полного исследователя лабиринта
-

Как устроен лабиринт

Лабиринт состоит из **ячеек** — маленьких комнат, у которых некоторые стены открыты.

```
+---+---+---+
|           |
+---+   +---+
|   |       |
+---+   +---+
|       |   |
+---+---+---+
```

У каждой ячейки 4 стороны: передняя, правая, задняя, левая. На некоторых сторонах стены, некоторые — открыты.

Робот начинает в одной ячейке и должен посетить каждую.

Робот знает: - Какие стены вокруг него (лидар) - Какие ячейки он уже посетил (память)

Робот не знает: - Как выглядит лабиринт впереди - Какого размера лабиринт

Обнаружение стен

Лидар может определить стены в любом направлении с помощью `check_wall()`:

```
distance, angle, quality = robot.lidar.check_wall(0)    # впереди
distance, angle, quality = robot.lidar.check_wall(90)   # справа
distance, angle, quality = robot.lidar.check_wall(180)  # сзади
distance, angle, quality = robot.lidar.check_wall(270)  # слева
```

В ячейке лабиринта 30 см стена находится примерно в 12-15 см.
Открытый проход — гораздо дальше (30 см и более).

Сканирование всех четырёх стен

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive()
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

# Проверяем все 4 направления
for name, angle in [('Перед', 0), ('Право', 90), ('Зад', 180),
                    ('Лево', 270)]:
    dist, _, quality = robot.lidar.check_wall(angle)

    if dist is not None and quality > 0.3 and dist < 0.25:
        print(f"{name}: СТЕНА на {dist*100:.1f}см")
    else:
        print(f"{name}: ОТКРЫТО")
```

```
robot.stop()
```

Попробуй: Поставь робота в ячейку лабиринта и запусти код. Он должен показать 3 стены и 1 проход (или как у тебя настроена ячейка).

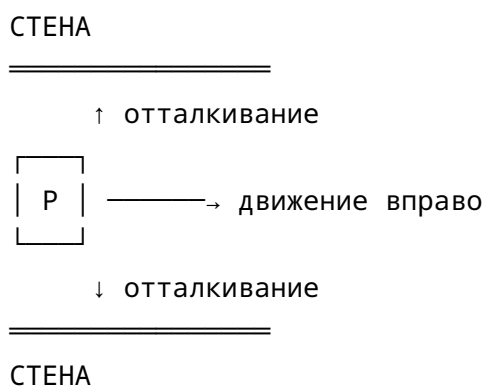
Перемещение между ячейками с `move_to_wall()`

Двигаться вслепую опасно — робот может врезаться в стену. Вместо этого используй `move_to_wall()`, который использует лидар для безопасного перемещения:

```
robot.move_to_wall(0)    # вперёд к следующей стене
robot.move_to_wall(270)  # влево к следующей стене
robot.move_to_wall(90)   # вправо к следующей стене
robot.move_to_wall(180)  # назад к следующей стене
```

Что делает `move_to_wall()`

Во время движения он постоянно: 1. **Следит за стеной впереди** — останавливается, когда достаточно близко 2. **Отталкивается от боковых стен** — держится по центру коридора 3. **Выравнивается по стенам** — держит робота прямо



Движение через проход

```
import time
from evobot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
```

```

robot.drive = MecanumDrive()
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

# Находим открытое направление
for name, angle in [('Перед', 0), ('Право', 90), ('Зад', 180),
('Лево', 270)]:
    dist, _, q = robot.lidar.check_wall(angle)
    if dist is None or q is None or q < 0.3 or dist > 0.25:
        print(f"{name} открыто! Двигаемся туда...")
        robot.move_to_wall(angle)
        break

robot.drive.halt()
robot.stop()

```

Что произойдёт: Робот найдёт проход, проедет через него и остановится в следующей ячейке, когда увидит стену впереди.

Упражнение 6.1: Проехать и осмотреться

Проедь через проход, затем осмотри стены в новой ячейке. Выведи то, что нашёл.

Решение

```

import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive()
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

# Находим проход и проезжаем через него
for name, angle in [('Перед', 0), ('Право', 90), ('Зад', 180),
('Лево', 270)]:
    dist, _, q = robot.lidar.check_wall(angle)
    if dist is None or q is None or q < 0.3 or dist > 0.25:
        print(f"Двигаемся {name}...")

```

```

        robot.move_to_wall(angle)
        break

time.sleep(0.5)

# Осматриваем новую ячейку
print("Новая ячейка:")
for name, angle in [('Перед', 0), ('Право', 90), ('Зад', 180),
('Лево', 270)]:
    dist, _, q = robot.lidar.check_wall(angle)
    if dist is not None and q is not None and q > 0.3 and dist <
0.25:
        print(f" {name}: СТЕНА на {dist*100:.1f}см")
    else:
        print(f" {name}: ОТКРЫТО")

robot.drive.halt()
robot.stop()

```

Запоминаем, где были

Чтобы исследовать лабиринт, роботу нужно помнить: - **Какие ячейки посетил** (чтобы не ходить по кругу) - **Где стены** (чтобы знать, какие ячейки соединены)

Используем простые структуры данных Python:

```

# Отслеживаем посещённые ячейки как координаты (x, y)
visited = set()

# Отслеживаем стены: (x, y, направление) -> True/False
walls = {}

```

Робот начинает в ячейке (0, 0). При движении: - **Вперёд (0)**: x увеличивается → (1, 0) - **Назад (180)**: x уменьшается → (-1, 0) - **Влево (270)**: y увеличивается → (0, 1) - **Вправо (90)**: y уменьшается → (0, -1)

```

        Вперёд (+x)
          ↑
Влево ← (0,0) → Вправо
(+y)      ↓      (-y)
        Назад (-x)

```

Запись стен

```
# Смещение координат по направлению
DIR_DELTA = {0: (1, 0), 90: (0, -1), 180: (-1, 0), 270: (0, 1)}
OPPOSITE = {0: 180, 180: 0, 90: 270, 270: 90}

visited = set()
walls = {}

def set_wall(x, y, direction, has_wall):
    """Записать стену (также записывает со стороны соседа)."""
    walls[(x, y, direction)] = has_wall
    # Сосед видит ту же стену с противоположной стороны
    dx, dy = DIR_DELTA[direction]
    walls[(x + dx, y + dy, OPPOSITE[direction])] = has_wall
```

Зачем записывать с обеих сторон? Если у ячейки (0,0) есть стена справа, значит у ячейки (0,-1) есть стена слева. Записав обе стороны, мы всегда знаем о стенах, даже до того, как посетим ячейку.

Выбор направления

Простейшая стратегия: **посетить любого непосещённого соседа.**

```
EXPLORE_PRIORITY = [270, 180, 90, 0] # лево, назад, право, вперёд

def get_next_cell(x, y):
    """Найти непосещённого соседа без стены на пути."""
    for direction in EXPLORE_PRIORITY:
        dx, dy = DIR_DELTA[direction]
        nx, ny = x + dx, y + dy

        # Проверяем: есть ли стена в этом направлении?
        has_wall = walls.get((x, y, direction))

        # Если нет стены И не посещено → едем туда!
        if has_wall is False and (nx, ny) not in visited:
            return direction, nx, ny

    return None # Нет непосещённых соседей
```

Порядок приоритетов определяет, какое направление робот предпочитает. Приоритет «сначала налево» приводит к методичному исследованию (как следование вдоль левой стены).

Упражнение 6.2: Две ячейки

Осмотри первую ячейку, переместись к непосещённому соседу, осмотри вторую ячейку.

Решение

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive()
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

DIR_DELTA = {0: (1, 0), 90: (0, -1), 180: (-1, 0), 270: (0, 1)}
OPPOSITE = {0: 180, 180: 0, 90: 270, 270: 90}
visited = set()
walls = {}

def set_wall(x, y, direction, has_wall):
    walls[(x, y, direction)] = has_wall
    dx, dy = DIR_DELTA[direction]
    walls[(x + dx, y + dy, OPPOSITE[direction])] = has_wall

# Начинаем в (0, 0)
x, y = 0, 0

for cell_num in range(2):
    print(f"\nЯчейка ({x}, {y}):")
    visited.add((x, y))

    # Сканируем все стены
    for name, angle in [('Перед', 0), ('Право', 90), ('Зад', 180), ('Лево', 270)]:
        dist, _, q = robot.lidar.check_wall(angle)
        has_wall = dist is not None and q is not None and q > 0.3
        and dist < 0.25
```

```

        set_wall(x, y, angle, has_wall)
        print(f" {name}: {'СТЕНА' if has_wall else 'ОТКРЫТО'}")

# Находим непосещённого соседа
for direction in [270, 180, 90, 0]: # лево, назад, право,
    # вперёд

    dx, dy = DIR_DELTA[direction]
    nx, ny = x + dx, y + dy
    if walls.get((x, y, direction)) is False and (nx, ny) not
in visited:
        dir_name = {0: 'Вперёд', 90: 'Вправо', 180: 'Назад',
270: 'Влево'}[direction]
        print(f" Двигаемся {dir_name} в ({nx}, {ny})")
        robot.move_to_wall(direction, max_travel=0.30)
        x, y = nx, ny
        break

robot.drive.halt()
robot.stop()

```

Полный исследователь лабиринта

Теперь собираем всё в цикл:

1. **Осмотреть** стены текущей ячейки
2. **Записать** стены и отметить ячейку как посещённую
3. **Выбрать** непосещённого соседа
4. **Переместиться** туда с помощью `move_to_wall()`
5. **Повторить**, пока есть непосещённые соседи

```

import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive()
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

# Вспомогательные словари
DIR_DELTA = {0: (1, 0), 90: (0, -1), 180: (-1, 0), 270: (0, 1)}
OPPOSITE = {0: 180, 180: 0, 90: 270, 270: 90}

```



```

DIR_NAMES = {0: "Вперёд", 90: "Вправо", 180: "Назад", 270:
"Влево"}

# Карта
visited = set()
walls = {}

def set_wall(x, y, direction, has_wall):
    walls[(x, y, direction)] = has_wall
    dx, dy = DIR_DELTA[direction]
    walls[(x + dx, y + dy, OPPOSITE[direction])] = has_wall

# Начинаем в (0, 0)
x, y = 0, 0

for step in range(100): # ограничение безопасности
    print(f"\n--- Ячейка ({x}, {y}) - Шаг {step + 1} ---")

    # 1. Сканируем стены
    visited.add((x, y))
    for angle in [0, 90, 180, 270]:
        dist, _, q = robot.lidar.check_wall(angle)
        has_wall = dist is not None and q is not None and q > 0.3
    and dist < 0.25
        set_wall(x, y, angle, has_wall)
        status = f"{dist*100:.1f}см СТЕНА" if has_wall else
"ОТКРЫТО"

        print(f" {DIR_NAMES[angle]:8s}: {status}")

# 2. Ищем непосещённого соседа (приоритет: лево, назад, право,
вперёд)
    next_move = None
    for direction in [270, 180, 90, 0]:
        dx, dy = DIR_DELTA[direction]
        nx, ny = x + dx, y + dy
        if walls.get((x, y, direction)) is False and (nx, ny) not
in visited:
            next_move = (direction, nx, ny)
            break

    if next_move is None:
        print(" Нет непосещённых соседей – готово!")
        break

```

```

direction, nx, ny = next_move
print(f" Двигаемся {DIR_NAMES[direction]} -> ({nx}, {ny})")

# 3. Поехали!
robot.move_to_wall(direction, max_travel=0.30)
x, y = nx, ny

print(f"\nИсследовано {len(visited)} ячеек!")
robot.drive.halt()
robot.stop()

```

Вот и весь исследователь лабиринта! Примерно 40 строк основной логики.

Упражнение 6.3: Добавь таймер

Добавь замер времени, чтобы узнать, сколько длилось исследование. Выведи общее время и среднее время на ячейку.

Решение

```

# Добавь перед циклом:
t_start = time.time()

# Измени финальный вывод на:
t_total = time.time() - t_start
print(f"\nИсследовано {len(visited)} ячеек за {t_total:.1f}с")
print(f"В среднем: {t_total/len(visited):.1f}с на ячейку")

```

Упражнение 6.4: Быстрее!

Заставь робота исследовать быстрее, передав более высокую скорость в `move_to_wall()`.

Решение

```

# Измени строку движения на:
robot.move_to_wall(direction, speed=0.5)

```

Робот едет быстрее, но останавливается безопасно — `move_to_wall()` всё время следит за стенами.

Внимание: Слишком высокие скорости (>0.8) могут быть чрезмерными для реакции лидара. Начни с 0.3 и увеличивай постепенно.

Как работает `move_to_wall()` внутри

Ты использовал `move_to_wall()` как чёрный ящик. Вот что происходит внутри:

```
# Упрощённая версия того, что делает move_to_wall:
while True:
    # 1. Читаем стену впереди
    distance, angle, quality = robot.lidar.check_wall(direction)

    # 2. Достаточно близко? Стоп!
    if distance <= stop_distance:
        robot.drive.halt()
        return

    # 3. Читаем ВСЕ остальные стены
    for wall_dir in [0, 90, 180, 270]:
        d, a, q = robot.lidar.check_wall(wall_dir)

        # Слишком близко к боковой стене? Отталкиваемся!
        if d < stop_distance:
            # Добавляем боковое смещение от стены

        # Используем угол стены для выравнивания
        # Усредняем углы всех стен для плавного поворота

    # 4. Задаём цель движения
    robot.drive.set_target_position(dx, dy, dtheta)
```

Ключевые идеи: - Читает стены непрерывно (не один раз) - Отталкивается от стен, к которым слишком близко - Выравнивается по всем видимым стенам одновременно - Использует `set_target_position()` для плавного управления моторами

Чего не хватает: возврат назад

У нашего исследователя есть одно большое ограничение: когда все соседи посещены, он останавливается. В большом лабиринте он может застрять в тупике, а непосещённые ячейки останутся в другом месте.

```
+---+---+---+
|   |       |
+   +---+   +
| P |   |   |   P застрял! Все соседи посещены.
+   +   +   +   Но правая верхняя ячейка ещё не посещена.
| . | . | . |
+---+---+---+
```

Решение: Когда застрял, возвращайся по пройденному пути, пока не найдёшь ячейку с непосещёнными соседями. Это называется **поиск в глубину (DFS)**.

Мы добавим возврат назад в следующей главе!

Итоги

Ты научился:

- Сканировать стены с `check_wall()` для определения стен и проходов
- Использовать `move_to_wall()` для безопасного перемещения между ячейками
- Отслеживать посещённые ячейки с помощью множества (`set`) Python
- Записывать стены с обеих сторон с помощью словаря (`dict`)
- Выбирать следующую ячейку по приоритету (исследование «сначала налево»)
- Строить полного исследователя лабиринта за ~40 строк

Ключевой паттерн исследования лабиринта:

```
while True:
    # Сканируем стены текущей ячейки
    # Записываем стены в карту
    # Находим непосещённого соседа
    # Перемещаемся туда с robot.move_to_wall(direction,
max_travel=0.30)
```

Что дальше?

В Главе 7 ты изучишь: - **Возврат назад** — возвращение в предыдущие ячейки, когда застрял - **Планирование пути** — поиск кратчайшего пути между ячейками - **Большие лабиринты** — навигация по лабиринтам 5x5 и 10x10